

Overview – YOLO

Context:

Image detection systems like **R-CNN**, **Fast/Faster R-CNN** that were used before **YOLO** made object detection in **two steps**. First, they proposed candidate **bounding boxes** containing potential **class objects**, and then they **classified** each box with a **CNN**. However, this process was too slow, because each image had many forward passes through the CNN.

Instead of generating region proposals, YOLO divides the image into a **grid** and makes **predictions for each cell**. YOLO treats detection as a **single regression problem**. It directly maps the **image pixels**, the **bounding boxes** and **class probabilities**. So instead of generating proposals, YOLO predicts everything at once, in one pass through the network, hence the name "You Only Look Once"

We will focus on the first released version of YOLO: YOLO1.

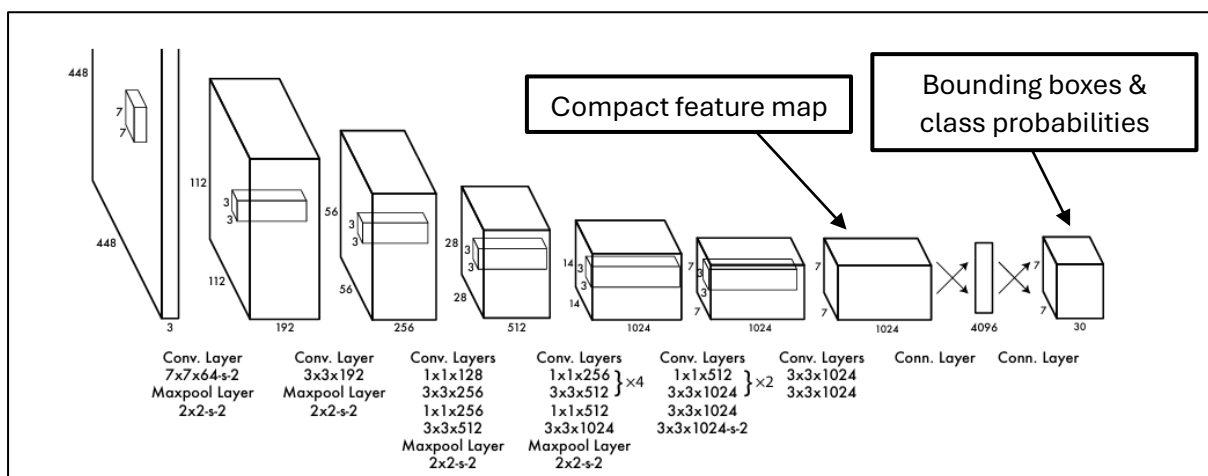
How does YOLO1 compute image detection?

YOLO1 computes image detection in a few steps:

1) Grid Division

- YOLO1 first takes an image as an input and resizes it to a **3D-tensor** with dimensions **$448 \times 448 \times 3$** (height = 448, width = 448, channels RGB = 3).
- YOLO1 uses a CNN to process the image through **24 convolutional layers** that slide **convolution filters/kernels** across the image to detect **local visual features** (e.g. edges, corners, color blobs). The CNN stacks multiple layers to capture **higher-level features** (object parts, shapes, semantic concepts), and after several layers and a **pooling process**, the image is transformed into a **compact feature map**, which is a **3D-tensor** with dimensions **$7 \times 7 \times 1024$** .

What's the process behind the 24 convolutional layers?



A) Input

The **input** of the CNN in YOLO1 is a $448 \times 448 \times 3$ tensor (Height $\times$ Width $\times$ RGB Channels).

B) First convolutional layer

The **first convolution layer** applies **64 kernels** that slide over the input image to extract features. Each kernel has dimensions of $7 \times 7 \times 3$, it has a **stride** equal to 2, which means that the kernel slides from 2 pixels at each iteration, and a **padding** is set to 5, which means that the kernels have now $(448 + 5) \times (448 + 5)$ pixels to slide over.

For instance, for a 5×5 input, if we set $\text{Padding} = 0$ and $\text{Stride} = 1$, the 64 filters of $3 \times 3 \times 3$ can only “fit” 3 times across the image:

1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1

However, if we set $\text{Stride} = 1$, we must add a border of zeroes, so the filter can slide across the entire input, including the edges:

0	0	0	0	0	0	0
0	1	1	1	1	1	0
0	1	1	1	1	1	0
0	1	1	1	1	1	0
0	1	1	1	1	1	0
0	1	1	1	1	1	0
0	0	0	0	0	0	0

Then each of the 64 filters slides on the entire image with stride 2 and padding 3, and outputs a formula to produce **one feature map** (so we get 64 total). The formula for output dimension (Height and Width) of a convolution layer is:

$$O = \left\lfloor \frac{W - F + 2P}{S} \right\rfloor + 1 = \left\lfloor \frac{448 - 7 + 2 \times 3}{2} \right\rfloor + 1 = \lfloor 223.5 \rfloor + 1 = 223 + 1 = 224$$

Where:

- W = input width
- F = filter size
- P = padding
- S = stride

So output is a tensor with dimensions **$224 \times 224 \times 64$** .

Regarding the weights (trainable parameters) of YOLO1, each filter has:

$$7 \times 7 \times 3 = 147 \text{ weights}$$

With 64 filters:

$$147 \times 64 = 9408 \text{ weights}$$

Plus 64 bias terms (one per filter):

$$9408 + 64 = 9472 \text{ trainable parameters}$$

C) First Max Pooling Layer

Pooling layers are non-learnable operations that **reduce the spatial dimensions (width and height)** of feature maps while keeping the number of channels the same. YOLO1 uses **Max Pooling**, meaning it takes the **maximum value** from a small region of the feature map.

Same as the first convolutional layer, each of the 64 filters slides on the entire image with stride 2 and padding 0, and outputs a formula to produce **one feature map**. The formula for output dimension (Height and Width) of a max pooling layer is the same:

$$O = \left\lfloor \frac{W - F + 2P}{S} \right\rfloor + 1 = \left\lfloor \frac{224 - 2 + 2 \times 0}{2} \right\rfloor + 1 = \lfloor 111 \rfloor + 1 = 111 + 1 = 112$$

So output is a tensor with dimensions **112 × 112 × 64**.

For instance, with the input 2×2 patch, the result of the **max pooling operation** on that patch would be:

$$O = \left\lfloor \frac{W - F + 2P}{S} \right\rfloor + 1 = \left\lfloor \frac{2 - 2 + 2 \times 0}{2} \right\rfloor + 1 = \left\lfloor \frac{0}{2} \right\rfloor + 1 = 0 + 1 = 1 \text{ dimension (scalar)}$$

$$\text{Max}_{\text{pooling}} \begin{pmatrix} 1.2 & 5.3 \\ 0.5 & 0.3 \end{pmatrix} = 5.3$$

That single number 5.3 becomes on pixel in the **output feature map**.

Then the window slides 2 pixels to the right, **repeating this across the entire map**, so the output is a **down sampled** (half-resolution) version of the input feature maps, but it keeps only the **most prominent activation** in each small neighborhood.

Moreover, **after each convolution** only (be careful, the operation in a max pooling layer is purely spatial), YOLO1 applies the **Leaky ReLU activation function**, introducing **non-linearity**, allowing the network to model complex functions:

$$f(x) = \begin{cases} x & x > 0 \\ 0.1x & x \leq 0 \end{cases}$$

D) Last convolutional layer

The **last convolutional layer** works the same and returns a tensor of shape $7 \times 7 \times 1024$, which encodes rich, high-level visual information about **where and what objects** might be in the image. To make it simple, the spatial grid 7×7 still preserves “where things are” roughly, and each cell’s 1024 values describe “what is there”.

E) First fully connected layer

After feature extraction, YOLO1 **flattens** the $7 \times 7 \times 1024$ feature map into a **1D-vector** with dimensions:

$$7 \times 7 \times 1024 = 50176 \text{ dimensions}$$

That vector is a long list of numbers, each representing a “feature activation” from a particular position and channel in the final convolutional layer.

Now, this vector goes into a **fully connected layer** with **4,096 neurons**, each of them computing:

$$y_i = \sum_{j=1}^{50176} w_{ij}x_j + b_i \quad \text{for } i \in \{1, \dots, 4096\}$$

Where:

- x_j = input feature value j (each neuron has **50,176 weights $w_{i,j}$** + **1 bias b_i**)
- w_{ij} = weight connecting input j to output neuron i
- b_i = bias term for neuron i
- y_i = output of neuron i

This layer is called “**fully connected**”, because every neuron is connected to all inputs.

In addition, this layer has:

$$\text{Weights} \times \text{Biases} = 50,176 \times 4,096 = 205,520,896 \text{ parameters}$$

This is one reason why YOLO1's fully connected layers are **very large and memory-heavy**. Later versions of YOLO1 remove these fully connected layers entirely and make the network **fully convolutional**.

Then the first fully connected layer also computes the same **Leaky ReLU function** on the output.

F) Second and final fully connected layer

The **second fully connected layers** takes a vector of size 4096 as an input. Each of the 4096 neurons also computes the same formula:

$$y_i = \sum_{j=1}^{4096} w_{ij}x_j + b_i \quad \text{for } i \in \{1, \dots, 1470\}$$

Once again, every neurons is connected to every input.

In addition, this layer has:

$$\text{Weights} \times \text{Biases} = 4096 \times 1470 = 6\,019\,200 \text{ parameters}$$

That's **much smaller than the first fully connected layer**, but it's still quite large.

Contrary to the first fully connected layer, this layer **does not use an activation function**, so the output values can freely **range across the real numbers**.

G) Output

The output vector of length **1470** is reshaped into a **3D tensor** $7 \times 7 \times 30$ according to that formula:

$$7 \times 7 \times 30 = 7 \times 7 \times (B \times 5 + C) = 1470 \text{ outputs}$$

Where:

- $B = 2$ = bounding boxes with coordinates and confidence
- $C = 20$ = class probabilities

This corresponds to dividing the image into a 7×7 , and each grid cell predicting **30 numbers** that encode both object localization and classification.

In other words, for each of the $7 \times 7 = 49$ grid cells, there are 30 outputs:

- **20 class probabilities:** one per class (for PASCAL VOC's 2012 20 classes)
- **2 bounding boxes**, each with 5 parameters:
 - x, y: coordinates of box center (relative to cell)
 - w, h: width and height (relative to full image)
 - C: confidence score (objectness $\times$ IoU with ground truth)

For instance, let's take a single output cell in the final 7×7 grid, its 30 outputs might look like this:

Index	Meaning	Example Value
0–4	Box 1 (x, y, w, h, conf)	[0.43, 0.58, 0.21, 0.19, 0.82]
5–9	Box 2 (x, y, w, h, conf)	[0.46, 0.57, 0.25, 0.23, 0.79]
10–29	Class probabilities	[0.0, 0.01, 0.85, 0.03, ...] $\rightarrow$ maybe “car” has highest = 0.85

All those 30 values are **directly output** by the last FC layer.

2) Predictions per Grid Cell

Each cell predicts:

- **B = 2 bounding boxes** characterized by:
 - (x, y): center of the box (relative to cell)
 - (w, h): width and height (relative to the image)
 - confidence score:

$$C_b = \text{Pr}(\text{object}) \times \text{IOU}_{\text{pred}}^{\text{truth}}$$

- **C = 20 class probabilities:**

$$\text{Pr}(\text{Class}_i \mid \text{Object})$$

The confidence score C_b measures both “**how likely**” the cell contains an object (the probability term $\text{Pr}(\text{object})$), and “**how well**” the predicted box matches the actual object (the quality term $\text{IOU}_{\text{pred}}^{\text{truth}}$):

- The term $\text{IOU}_{\text{pred}}^{\text{truth}}$ is the **ground-truth object**, which is the real, correct object information that we know from the **training dataset annotations**. During training, YOLO computes the ground-truth like so:

$$\text{IOU}_{\text{pred}}^{\text{truth}} = \frac{\text{Area}(B_{\text{pred}} \cap B_{\text{truth}})}{\text{Area}(B_{\text{pred}} \cup B_{\text{truth}})}$$

Where:

- Predicted bounding box = B_{pred}
- Ground-truth bounding box = B_{truth}
- If the grid cell contains the center of a **ground-truth object**, we set $Pr(Object) = 1$, otherwise, $Pr(Object) = 0$.

For every class and every bounding box, YOLO1 computes the **final confidence score** as:

$$\begin{aligned} Y &= P(Class_i | Object) \times C_b \\ &= P(Class_i | Object) \times P(Object) \times IOU_{Prediction}^{Truth} \\ &= P(Class_i) \times IOU_{Prediction}^{Truth} \end{aligned}$$

YOLO will keep the highest of these per class and per box after thresholding.

For instance, for a single grid cell that has:

- **Box 1** predicted $C_1 = 0.8$
- **Box 2** predicted $C_2 = 0.5$

And its **class probabilities** are:

- $P(Dog|Object) = 0.9$
- $P(Car|Object) = 0.05$
- $P(Person|Object) = 0.03$

Then the **final score** in Box 1 is:

- For “dog”: $0.9 \times 0.8 = 0.72$
- For “car”: $0.05 \times 0.8 = 0.04$

And the final score in Box 2 is:

- For “dog”: $0.9 \times 0.5 = 0.45$

YOLO will keep the **highest of these per class and per box after thresholding**, which is “dog” for both boxes in that cell.

3) Post-processing (filtering and merging)

After YOLO1 decoded all $7 \times 7 \times 2 = 98$ boxes, and **computed the final class scores**, it **discards boxes** where confidence $<$ threshold, and it applies a **Non-Max Suppression (NMS)** that consists in:

- **Sorting** boxes by confidence for each class separately.
- **Removing** boxes that overlap highly ($IOU > 0.5$) with a higher-scoring box.
- **Keeping** only one box per object.

The remaining boxes are the **final detections**, each with x, y, w, h, class label, and final confidence.

In a nutshell, these confidence scores reflect how confident the model is that the box contains an object and also how accurate it thinks the box is that it predicts. For each grid cell, if no object center falls inside, all B boxes are trained to predict no object (confidence = 0), but if one object center falls inside, the system finds which of the B boxes best matches the ground truth (highest IoU), it assigns that box to predict that object (with real class and coordinates), and the other boxes still predict “no object”. That’s why in YOLOv1, each cell can be responsible for at most one object, even though it outputs B boxes.